Cleaning the Trading FIX message data

JSE Trading Operations conformance test messages, July to September 2026 · cleaning run of 01/11/2026

The 36 Excel exports of the Trading FIX message data hold **3,763,516 rows** across 11 message types. After cleaning they form one table of **2,169,820 distinct messages**, one row per message, with times and numbers stored as times and numbers. 1,593,696 export rows (42.3%) were repeated copies of a message and were merged. 5,611 messages that had no CompID were given one from related messages, and 232,316 client requests were matched to the JSE order they refer to.

Language and tools

The cleaning was first written in **Python**, in the script `silver.py`, with these libraries:

Library	What it does in the cleaning
pandas	A Python library for tables. It cleans the rows of each Excel file: renames headers, trims values and converts times and numbers.
DuckDB	A database that runs inside the Python program and is given instructions in SQL (Structured Query Language). It combines the files, merges repeated rows, fills missing CompIDs and links requests to orders.
python-calamine	Reads the Excel files.
pyarrow	Writes the cleaned table as a Parquet file, a compressed file format for tables.

The scripts run on a computer, not inside the cloud storage service (Supabase Storage). Each script downloads its input files from the storage bucket, does the work on the computer, and uploads the result. The storage only holds the files.

The same rules were later written a second time in **T-SQL**, the SQL language of Microsoft SQL Server, as stored procedures (sets of SQL statements saved in the database under a name). The procedure `silver.build_silver` does the cleaning, and the script `05_check_counts.sql` compares its row counts with those of the Python run.

Before cleaning: CompIDs replaced

Before any other step, every real CompID in the Excel files is replaced with its fictitious CompID from a key file. A CompID is the login name of one client connection to the JSE trading gateway. The program then searches the whole file for any real CompID and stops without saving the file if it finds one. Every CompID in the cleaned table is fictitious.

Cleaning steps

1. Combine the files of each message type

The files of one message type, for all three months, are combined into one table, with every value kept as text. Each file's message type is taken from the values in its Message Type column, not from its file name, because the file `July_2026/Logout [B].xlsx` holds Logon Response messages. The column `source_file` records the file each row came from.

2. Standard field names

Headers become lower-case names with underscores. Headers that name one field differently in different files are given one name.

```
Client Order ID -> client_order_id
Original Client Order ID -> orig_client_order_id
OrderBook -> order_book
TypeofTrade -> type_of_trade
second 'Message Type' column in Reject -> ref_message_type
```

3. Trim values and mark empty values

Spaces and tabs at the start and end of each value are removed. A value that is then empty is stored as empty (NULL, the database's mark for no value). Other unprintable characters are kept, because some Self Trade Prevention Keys contain them.

4. Convert times

The exports write a 24-hour time followed by am or pm. The am or pm is removed and the text converted to a date and time. A time field is converted only if at least 95% of its values convert; a value that does not convert becomes empty.

```
2026-07-01 20:15:01.7941 pm -> 2026-07-01 20:15:01.7941
```

5. Convert numbers, keep codes as text

Thousands commas are removed. A field is converted to numbers only when every filled value in it is a number. Fields that hold IDs or codes (names containing id, code, type, status, side, key and similar) stay text, so leading zeros and letters are kept.

```
14,340.0000 -> 14340.0000 (limit_price, decimal)
00178573862373600000 stays text (client_order_id)
```

6. Put all message types in one table

The 11 tables are stacked into one table. A field that a message type does not have is empty on that type's rows. Fields with a short name that means different things in different types are renamed: Status → mass_cancel_status, Reason → logout_reason, the CompID inside a Logon → logon_compid. The network capture fields (source, destination, bytes, sequence_no, packet_id) and the two fields empty in every row (expire_time, segment) are left out.

7. Merge repeated rows

Many messages are recorded more than once. Rows that are identical in every field, capture time included, are merged into one, and the field `copies` gives the number of export rows merged. Copies of one execution report differ in capture time and in whether the CompID is filled, so execution reports are merged by Execution ID instead, a JSE ID unique to each report: one row per Execution ID, with the earliest capture time and a filled CompID if any copy has one. The program stops with an error if two rows with one Execution ID differ in any other field.

8. Remove 'not set' placeholders

The exports use -1 to mean 'not set'. In reject_code, minimum_quantity, stop_price and password_expiry, -1 is replaced with an empty value, so that it is not counted as a real code, quantity or price.

9. Fill missing CompIDs

Some JSE replies have no CompID. An empty CompID is filled from other messages with the same Order ID, then from messages with the same Client Order ID, only where that ID is used by a single CompID. Order ID NONE, which the JSE writes when it cannot find the order, is not used. `comp_id_filled` is true on the 5,611 messages filled this way.

10. Link client requests to orders

New orders, cancel requests and amend requests often carry no JSE Order ID. Such a request is given the Order ID of the first execution report or order cancel reject with the same CompID and Client Order ID captured within 1 second after it. `order_id_linked` is true on the 232,316 requests linked this way.

11. Number the messages

Each message is given a `message_id`, numbered in order of CompID, capture time, message type and Execution ID, and a `date` field taken from the capture time.

Rows before and after cleaning

Message type	Code	Export rows	Messages after cleaning	Repeated rows merged	Share repeated
Logon	A	74,580	55,557	19,023	25.5%
Logon Response	B	23,846	19,947	3,899	16.4%
Logout	5	21,306	15,617	5,689	26.7%
New Order	D	325,344	205,175	120,169	36.9%
Order Cancel Request	F	539,586	318,157	221,429	41.0%
Order Cancel/Replace Request	G	929,951	558,305	371,646	40.0%
Order Mass Cancel Request	q	354	246	108	30.5%
Execution Report	8	699,162	245,298	453,864	64.9%
Order Cancel Reject	9	21,909	14,601	7,308	33.4%
Order Mass Cancel Report	r	464	309	155	33.4%
Reject	3	1,127,014	736,608	390,406	34.6%
Total		**3,763,516**	**2,169,820**	**1,593,696**	**42.3%**

What the cleaning does not change

• Fields missing from some months' files stay empty for those months. September new orders have no side, quantity, price or time in force; execution reports have Execution Type only in August and September and Order Status only in July and September.

• Timestamp is South African time (UTC+2) and Transact Time is UTC, 2 hours behind. Both are kept in their own time zone.

• -1.0000 in limit_price is kept as a price; a query on limit prices must leave it out.

• Values outside the JSE code lists are kept: capacity 0, and security_id 0 on every mass cancel request.

• CompIDs that cannot be filled stay empty, including on Logon Response and Logout messages, which carry no order ID.